I costrutti if, for, while e i registri TRIS e PORT

Lezione 3 — 3MAK, SST-INF-01c — Prendere decisioni, ripetere, comandare i piedini del PIC

08/10/2026

1 Dove eravamo

Finora abbiamo visto:

- le **variabili** e i loro **tipi** (`char`, `int`, `float`) e gli **operatori** (dispensa A01);
- gli **operatori logici** e gli **operatori bitwise** (dispensa A02).

Oggi vediamo i costrutti: i "comandi" che decidono **quali** istruzioni eseguire e **quante volte**. Sono tre, e vanno imparati bene:

1. il costrutto **decisionale**: **`if`** (con la variante `if-else`);
2. due costrutti di **ripetizione**: **`for`** e **`while`**.

Alla fine vediamo come il PIC decide se un **piedino** è un **ingresso** o un'uscita: i registri **TRIS** e **PORT**.

2 Il costrutto decisionale: if

Definizione

Il costrutto **if** ("se") permette al microcontrollore di **prendere una decisione**: **eseguire o non eseguire** un pezzo di codice, in base a una condizione.

Esempio

Se piove, prendo l'ombrello. Devi uscire di casa: se piove prendi l'ombrello, altrimenti no. È una decisione. Oppure: se piove prendo l'ombrello, **altrimenti** prendo gli occhiali da sole: in questo caso c'è qualcosa da fare in entrambi i casi.
In un circuito: "se non sto premendo il pulsante, non accendere il **LED**".

2.1 La forma semplice

```
if (condizione)
{
    // codice da eseguire SE la condizione e' vera
}
```

Si scrive **`if`**, si apre la parentesi **tonda**, si scrive la **condizione**, si chiude la tonda; seguono due parentesi **graffe**, una aperta e una chiusa. Dentro le graffe va tutto il codice da eseguire **se la condizione è vera**.

Nota

Il codice viene sempre eseguito **dall'alto verso il basso**. Quando il programma arriva all'**`if`**, decide: se la condizione è **vera** esegue il codice tra le graffe; se è **falsa** lo **salta**. In entrambi i casi poi prosegue con le istruzioni dopo la graffa chiusa.

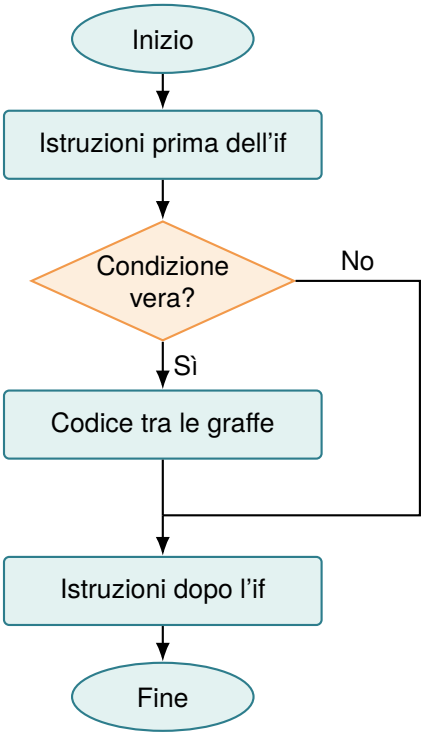


Figura 1: Diagramma di flusso dell'`if` semplice: se la condizione è falsa, il codice tra le graffe viene saltato.

2.2 La forma con else

```
if (condizione)
{
    // codice da eseguire SE la condizione e' vera
}
else
{
    // codice da eseguire SE la condizione e' falsa
}
```

Definizione

`else` significa "**altrimenti**". Con `if-else` il programma non sceglie solo se fare o no una cosa: **sceglie tra due cose**. Se la condizione è vera esegue il primo blocco, se è falsa esegue il secondo. Uno dei due blocchi viene **sempre** eseguito, mai entrambi.

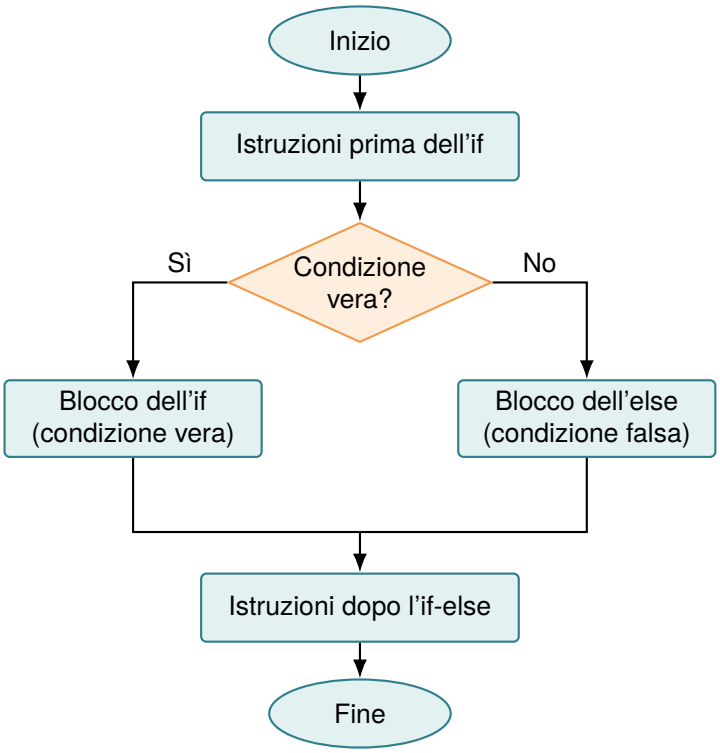


Figura 2: Diagramma di flusso dell'`if-else`: viene eseguito sempre uno solo dei due blocchi.

Nota

Quale usare? Nessuna delle due forme è "migliore": dipende da cosa bisogna fare. Se c'è qualcosa da fare solo quando la condizione è vera, basta l'**`if`**; se c'è da fare una cosa quando è vera e un'altra quando è falsa, serve l'**`if-else`**.

2.3 Le condizioni

Definizione

La **condizione** è una qualunque espressione che dia come risposta **vero** o **falso**: ad esempio "a uguale a 5" (`a == 5`), "b uguale a 7" (`b == 7`). Esempi scritti alla lavagna:

`a < 7`	a è minore di 7
`z > b`	z è maggiore di b
`a == b`	a è uguale a b (**doppio uguale**: è un **confronto**)
`z != 0`	z è diverso da zero
`x % 2 == 0`	il resto della divisione di x per 2 è zero: x è **pari**
`x % 2 != 0`	il resto non è zero: x è **dispari**

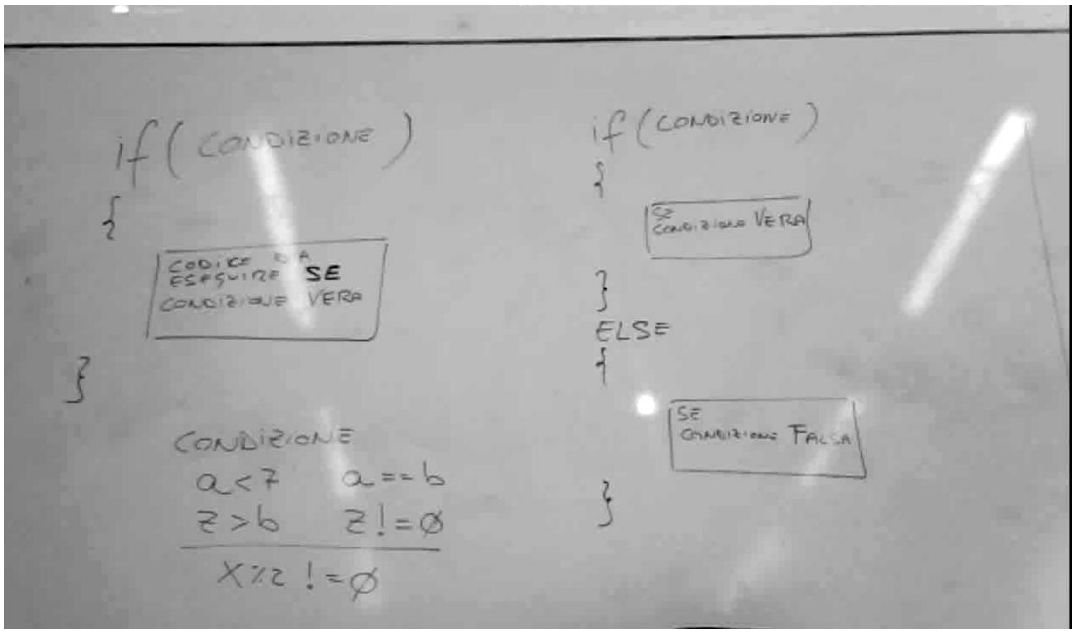


Figura 3: La lavagna (contrasto aumentato): a sinistra l'`if` semplice con il riquadro "codice da eseguire se condizione vera"; a destra l'`if-else` con i blocchi "se condizione vera" e "se condizione falsa"; in basso gli esempi di condizione `a<7`, `a==b`, `z>b`, `z!=0`, `x%2!=0`.

Attenzione

Nelle condizioni si usa il **doppio uguale** ==, che **confronta**. Il singolo uguale = invece **assegna** un valore a una variabile: sono due cose diverse.

Esempio

L'operatore `%` (**modulo**, visto nella dispensa A02) calcola il **resto** della divisione. `x % 2 == 0` è vero quando il resto della divisione per 2 è zero, cioè quando x è **pari** (es. $x = 6$: $6 : 2 = 3$ con resto 0). Alla lavagna è stata scritta anche la versione `x % 2 != 0`, vera quando x è **dispari** (es. $x = 7$: $7 : 2 = 3$ con resto 1).

```
if (x % 2 == 0)
{
    // eseguito solo se x e' pari
}
```

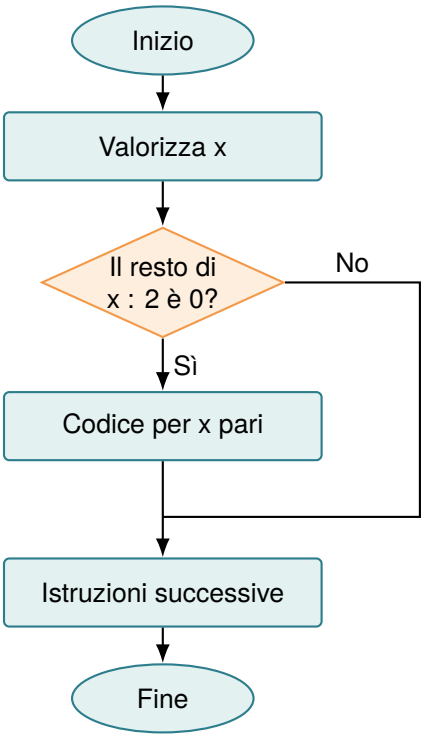


Figura 4: Diagramma di flusso dell'esempio "se x è pari".

3 I costrutti di ripetizione: for e while

Definizione

I costrutti di ripetizione (**cicli**) servono a ripetere un pezzo di codice più volte. Se un **LED** deve lampeggiare 10 volte, invece di scrivere "accendi, spegni, accendi, spegni…" dieci volte, si scrive una volta sola "accendi e spegni" e si dice al microcontrollore di ripeterlo 10 volte.

Nota

Due costrutti che fanno la stessa cosa. `for` e `while` fanno **esattamente la stessa cosa**: si potrebbe usare sempre solo uno dei due. Perché impararli entrambi? Perché i programmatori, per abitudine e per chiarezza, preferiscono:

- **for** quando si sa **esattamente quante volte** ripetere (es. un allarme che lampeggia sempre 10 volte);
- **while** quando **non si sa a priori** quante volte ripetere, perché dipende da cosa succede fuori (es. un robot cammina finché non arriva al bordo del tavolo: quanti passi sono? Non si sa prima).

È solo un consiglio tra programmatori, non un obbligo.

3.1 Gli ingredienti di un ciclo

Ogni ciclo che conta le ripetizioni ha bisogno di tre cose:

1. una **variabile** con uno **stato iniziale**: è il "contatore" che tiene il conto di quante volte abbia-

mo ripetuto. Si crea come una qualunque variabile (es. `int i = 0`). Il **nome** è libero; per convenzione si usa **`i`**, che sta per **indice**.

2. una **condizione di ripetizione**: finché è vera, il ciclo continua (es. `i < 10`);
3. un modificatore: l'istruzione che fa **avanzare** il contatore a ogni giro (es. `i = i + 1`).

Esempio

I salti. Qualcuno fa 10 salti e un altro li conta: quanti sono? Il numero che si conta "va salvato in una variabile": è proprio il **contatore** del ciclo, che tiene il conto di dove si è arrivati.
Da quanto si parte a contare? Da **zero**: 0 è più piccolo di 1. In programmazione si parte quasi sempre da 0.
Il **modificatore** decide come avanza il conteggio a ogni "salto": `i = i + 1` conta 0, 1, 2, 3, 4…; `i = i + 2` conta 0, 2, 4, 6…; `i = i + 3` conta 0, 3, 6, 9…
Come funziona **`i = i + 1`**: se `i` vale 0, calcolo $0 + 1 = 1$ e lo rimetto dentro `i`, che adesso vale 1. Al giro dopo $1 + 1 = 2$, poi 3, 4, 5… A ogni giro il contatore cresce di uno.

3.2 Il ciclo for

```
for (int i = 0; i < 10; i = i + 1)
{
    // codice da ripetere 10 volte
}
```

Definizione

Nella parentesi tonda del **`for`** ci sono **tre parti**, separate da **punto e virgola**:

```
for (int i = 0; i < 10; i = i + 1)
```

- in **rosso** lo **stato iniziale**: crea la variabile `i` e le dà il valore 0;
- in **verde** la **condizione di ripetizione**: si ripete finché `i` è minore di 10;
- in **blu** il **modificatore**: a ogni giro `i` aumenta di 1.

Tra le graffe va tutto quello che si vuole **ripetere**: una sola istruzione o cento, a scelta.

Esempio

Il for passo per passo.

1. Il programma arriva al `for`: **crea** `i` e le dà il valore **0**.
2. Si domanda (verde): `i < 10`? $0 < 10$, sì: esegue il codice tra le graffe.
3. Arrivato alla graffa chiusa, torna su ed **esegue** il modificatore (blu): `i` diventa **1**.
4. Si domanda: $1 < 10$? Sì: esegue di nuovo il codice. Poi `i` diventa 2, $2 < 10$, e così via.
5. Quando `i` vale **9** esegue la **decima** ripetizione; poi `i` diventa **10**.
6. Si domanda: $10 < 10$? **No** (10 non è minore di 10): il ciclo **finisce** e il programma prosegue con il codice dopo le graffe.

Risultato: il codice è stato eseguito **10 volte**, con `i` che vale 0 alla prima ripetizione, 1 alla seconda, . . . , 9 alla decima. Per ripetere 4 volte basta scrivere **`i < 4`**.

Attenzione

Con `i` che parte da 0, la condizione `i < 10` fa **10** ripetizioni, non 9 e non 11: i valori sono 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. È un punto su cui è **facile sbagliarsi**: contate sempre i giri a partire da **zero**.

3.3 Il ciclo while

```
// stato iniziale
int i = 0;
// condizione di ripetizione
while (i < 10)
{
    // codice da ripetere

    // modificatore (consigliato alla fine)
    i = i + 1;
}
```

Definizione

Il **while** (“finché”) ripete quello che c’è tra le graffe **finché la condizione è vera**. Gli ingredienti sono **gli stessi** del `for`, solo in posizioni diverse:

- lo **stato iniziale** si scrive **prima** del `while` (`int i = 0;`);
- la **condizione di ripetizione** va nella parentesi tonda del `while`;
- il **modificatore** va **dentro le graffe**, dove si vuole, ma è consigliato **alla fine**.

Il funzionamento è identico: con questi valori anche il `while` ripete 10 volte.

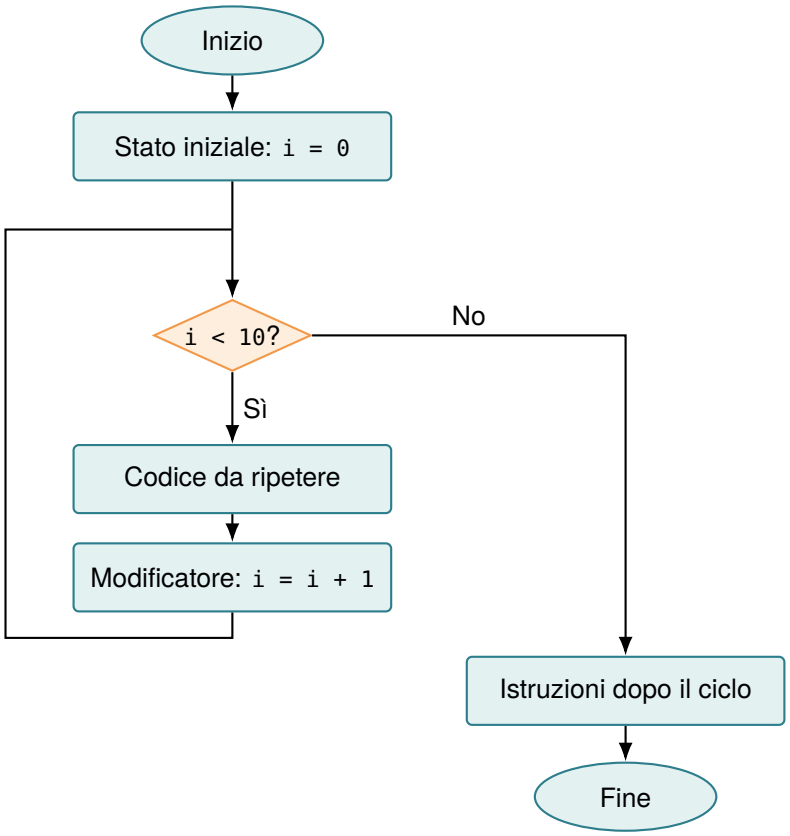


Figura 5: Diagramma di flusso comune a `for` e `while`: stato iniziale, domanda sulla condizione, codice, modificatore e ritorno alla domanda. Quando la condizione diventa falsa si esce dal ciclo.

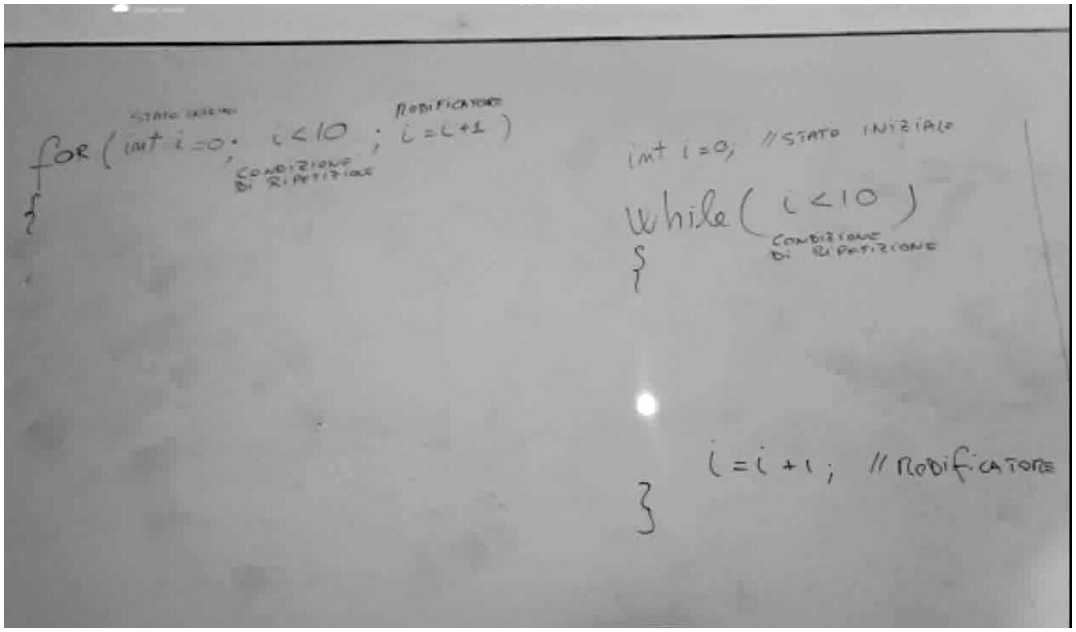


Figura 6: La lavagna: a sinistra il `for` con le tre parti etichettate (stato iniziale `int i=0`, condizione di ripetizione `i<10`, modificatore `i=i+1`); a destra lo stesso ciclo scritto con il `while`: stato iniziale prima, condizione nella tonda, modificatore in fondo alle graffe.

Nota

Perché il while quando non si sa quante volte? Nella **condizione** del `while`, invece di `i < 10`, si può scrivere "finché il dito è sul pulsante", "finché il sensore non è arrivato a fine corsa", "finché non succede qualcosa". Il sensore va **letto**, e dentro le graffe del `while` c'è lo spazio per farlo; scriverlo tutto dentro la parentesi del `for` sarebbe brutto da vedere. Si potrebbe fare anche con il `for`, non succede niente, ma normalmente si usa il **`while`**.
Quindi: **so quante volte** → `for`; **non lo so** → `while`.

4 Una curiosità: le classi (che in C non ci sono)

Nota

In C, quello che c'è da sapere è quasi tutto qui: i tipi di variabili, gli **operatori** (aritmetici, di confronto, logici, bitwise) e i **costrutti**. Altri linguaggi, come il **C++** e il **C#** ("C sharp"), hanno anche le **classi**.
Una **classe** permette di creare un "concetto" che **raggruppa** tante informazioni insieme. Esempio: per gestire al computer gli allievi, ogni allievo ha un nome e un cognome, una data di nascita, il colore degli occhi e dei capelli, un voto in italiano e uno in programmazione firmware, il voto di condotta, la taglia delle scarpe, un certo numero di penne e di quaderni... Invece di gestire tante **variabili separate**, si crea la classe "allievo" con tutte queste caratteristiche, e poi si crea un allievo con il suo nome e i suoi dati. Il vantaggio: se si sposta l'allievo da una classe all'altra, si spostano **tutte** le sue informazioni insieme, in un colpo solo. In C questo non c'è, e nel corso non lo useremo.

5 Ingressi e uscite: da Arduino al PIC

5.1 pinMode, digitalWrite, digitalRead

Esempio

Il supermercato. In un supermercato ci sono le porte da cui si **entra** e, dopo le casse, il punto da cui si **esce**: formalmente non si può entrare dall'uscita né uscire dall'entrata. Qualcuno ha deciso quale porta è l'entrata e quale l'uscita.
Il microcontrollore ha dei **piedini**, e ciascun piedino è una "porta" da cui l'informazione può entrare o uscire. Continuando l'analogia:

- **`pinMode`** decide quale porta è un'**entrata** e quale un'**uscita**;
- **`digitalWrite`**, su una porta d'uscita, decide se **far uscire una persona** oppure no;
- **`digitalRead`**, su una porta d'entrata, **fa entrare** qualcuno, cioè legge chi arriva da fuori.

Definizione

Su **Arduino** (che avete già usato):

- **`pinMode`** decide se un piedino è un'**entrata** (`INPUT`) o un'**uscita** (`OUTPUT`);

- **`digitalWrite`**, su un'uscita, decide se "far uscire" un 1 o uno 0;
- **`digitalRead`**, su un ingresso, legge se "entra" un 1 o uno 0.

Sul **PIC** questi comandi **non ci sono**: la stessa cosa si fa in un altro modo, con i **registri**.

5.2 I bit TRIS e PORT

Attenzione

Questa è, parole del docente, "**la cosa più importante**", "**il cuore del corso**": tutto quello che faremo con il PIC (accendere LED, leggere pulsanti) passa da **TRIS** e **PORT**.

Definizione

Ogni piedino del PIC è comandato da **due bit**:

- il bit **TRIS** decide se il piedino è un **ingresso** o un'**uscita**:
  - **0** → **uscita** (output);
  - **1** → **ingresso** (input);
- il bit **PORT** contiene il **valore** del piedino, 0 o 1: nell'analogia, se dalla porta esce (o entra) una persona oppure no.

Esempio

Come ricordarlo: lo **0** somiglia alla **O** di **O**utput, l'**1** somiglia alla **I** di **I**nput.

Definizione

Scrivere o leggere PORT.

- Se il piedino è un'**uscita**, PORT si **scrive**: scrivere 1 o 0 vuol dire imporre che su quel piedino ci sia 1 o 0.
- Se il piedino è un **ingresso**, PORT si **legge**.

Come si capisce se è una scrittura o una lettura? Dalla posizione rispetto all'**uguale**:

- **PORT a sinistra** dell'uguale → **scrittura** (per gli output);
- **PORT a destra** dell'uguale → **lettura** (per gli input).

Alla lavagna: `PORT = ...;` è una **scrittura**, `X = PORT;` è una **lettura**. Con i nomi veri dei registri del PIC16F628A:

```
// scrittura: PORTB a sinistra dell'uguale (output)
// bit 0-3 a 1: RB0, RB1, RB2, RB3 a livello alto
PORTB = 0b00001111;
// lettura: PORTA a destra dell'uguale (input)
x = PORTA;
```

Attenzione

Il valore scritto in **PORTB** riguarda **tutti e 8** i piedini insieme, un bit per piedino: `PORTB = 1` **non** accende tutta la porta, porta a 1 **solo il bit 0** (RB0) e mette a 0 **tutti gli altri**. Per questo conviene scrivere il valore in **binario** (`0b...`), dove si vede **bit per bit** cosa succede.

5.3 Perché i registri sono più veloci di digitalWrite

Esempio

Con `digitalWrite` di Arduino, per ogni piedino da comandare bisogna scrivere un'istruzione: per comandare **8** piedini servono **8** `digitalWrite`. Con i registri del PIC, una **sola** istruzione (es. `PORTB = ...`, vedi sotto) scrive tutti e 8 i bit in un colpo solo. A parità di quarzo, la scrittura diretta sul registro è quindi **almeno 8 volte più veloce** (in realtà anche di più, perché ogni `digitalWrite` esegue al suo interno diverse istruzioni di controllo).

5.4 I registri a 8 bit: TRISA, TRISB, PORTA, PORTB

Definizione

I bit non si gestiscono uno alla volta: sono organizzati in **byte**. Esiste quindi una variabile a **8 bit** (8 caselle) chiamata **TRISA**, una casella per ciascun piedino della porta A (RA0...RA7), e lo stesso vale per la porta B con **TRISB** (RB0...RB7). Scrivendo un numero in **TRISA** si decide, per **tutti** i piedini della porta A insieme, chi è ingresso e chi è uscita. Allo stesso modo **PORTA** e **PORTB** contengono i valori degli 8 piedini.

	7	6	5	4	3	2	1	0	
TRISA	x	x	x	x	x	1	1	x	RA1, RA2 = pulsanti → **1** (input)
TRISB	x	x	x	x	0	0	0	0	RB0–RB3 = LED → **0** (output)

Figura 7: La configurazione dei registri TRIS per la nostra scheda (dispensa A04 di elettronica). Le caselle sono numerate dal bit 7 (a sinistra) al bit 0 (a destra); x = indifferente, "a libera scelta".

Esempio

La nostra scheda. Sulla scheda disegnata in elettronica:

- i **pulsanti** sono su **RA1** e **RA2**. L'informazione arriva da fuori ed entra nel PIC: sono **input**, quindi in TRISA i bit 1 e 2 valgono **1**. Gli altri bit di TRISA sono a libera scelta;
- i **LED** sono su **RB0–RB3**. L'informazione va dal PIC verso l'esterno (la luce arriva ai nostri occhi): sono **output**, quindi in TRISB i primi 4 bit (0–3) valgono **0**. Gli altri bit non ci interessano.

Scritti questi valori, il PIC è configurato per accendere i LED e leggere i pulsanti. Come scriverli nel codice lo vedremo nei programmi della prossima settimana.

Nota

RB6 e RB7 “vivono di vita loro”. RB6 e RB7 sono i piedini usati dal **programmatore** (PGC e PGD, il clock e i dati dell’ICSP): mentre il PIC viene programmato, la loro configurazione è decisa dal sistema di programmazione, non da noi. Si possono comunque usare anche per altro: ad esempio si può mettere un LED su RB7 e configurarlo come **uscita**. Nei primi istanti, mentre il PIC viene programmato, quel piedino lavora per il programmatore; una volta programmato, la macchina parte e il piedino funziona come uscita.

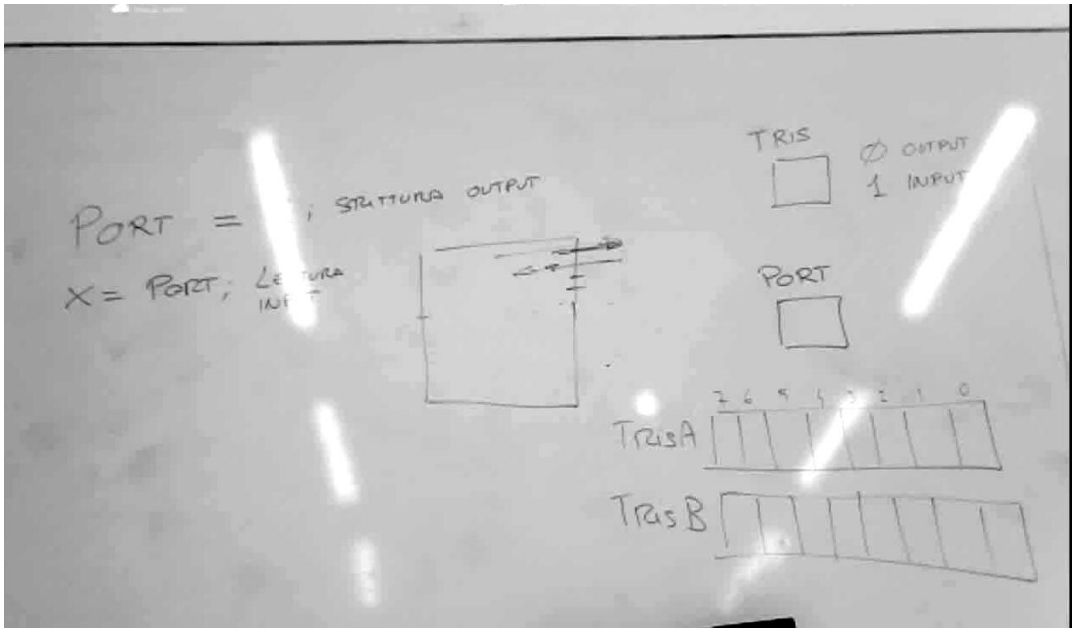


Figura 8: La lavagna: `PORT = ...;` (scrittura, output) e `X = PORT;` (lettura, input); il piedino con le frecce in entrata e in uscita; TRIS con 0 = output e 1 = input; i registri TRISA e TRISB a 8 caselle (bit da 7 a 0).

6 Verso la verifica

Attenzione

A breve ci sarà una **verifica conoscitiva**: non si chiede di scrivere codice, ma di **conoscere i concetti**. Bisogna sapere:

- i **tipi di dato** (`char`, `int`, `float`);
- gli **operatori**: più, meno, per, diviso, modulo; maggiore, minore, diverso, uguale; AND, OR, NOT logici; gli operatori **bitwise**;
- i **costrutti** `if`, `for`, `while`: come sono fatti, a cosa servono, cosa fanno e cosa non fanno.

La verifica è **sempre e solo** su quanto c’è **scritto nelle dispense**: quello che non c’è nella dispensa non c’è nella verifica. La dispensa può contenere anche argomenti utili non affrontati a voce in classe: vanno studiati comunque. Servirà la **calcolatrice**.

7 Cosa ricordare di questa lezione

- `if (condizione) { ...}`: esegue il blocco **solo se** la condizione è vera; con **`else`** si sceglie tra due blocchi.
- Nelle condizioni il **confronto** si scrive **==** (doppio uguale); `x % 2 == 0` vuol dire "x è pari".
- `for` e `while` fanno la stessa cosa: **`for`** quando si sa quante volte ripetere, **`while`** quando non si sa.
- Un ciclo ha tre ingredienti: **stato iniziale**, **condizione di ripetizione**, **modificatore**. Con `i = 0` e `i < 10` si fanno 10 giri (i da 0 a 9).
- Nel PIC: **TRIS** decide ingresso/uscita (**0 = Output**, **1 = Input**), **PORT** contiene il valore; PORT a sinistra dell'uguale = scrittura, a destra = lettura.
- TRISA/TRISB e PORTA/PORTB sono **registri a 8 bit**: una sola istruzione comanda 8 piedini (**più veloce** di 8 `digitalWrite`).
- Nella nostra scheda: RA1, RA2 **input** (TRISA bit 1 e 2 a 1); RB0–RB3 **output** (TRISB bit 0–3 a 0).